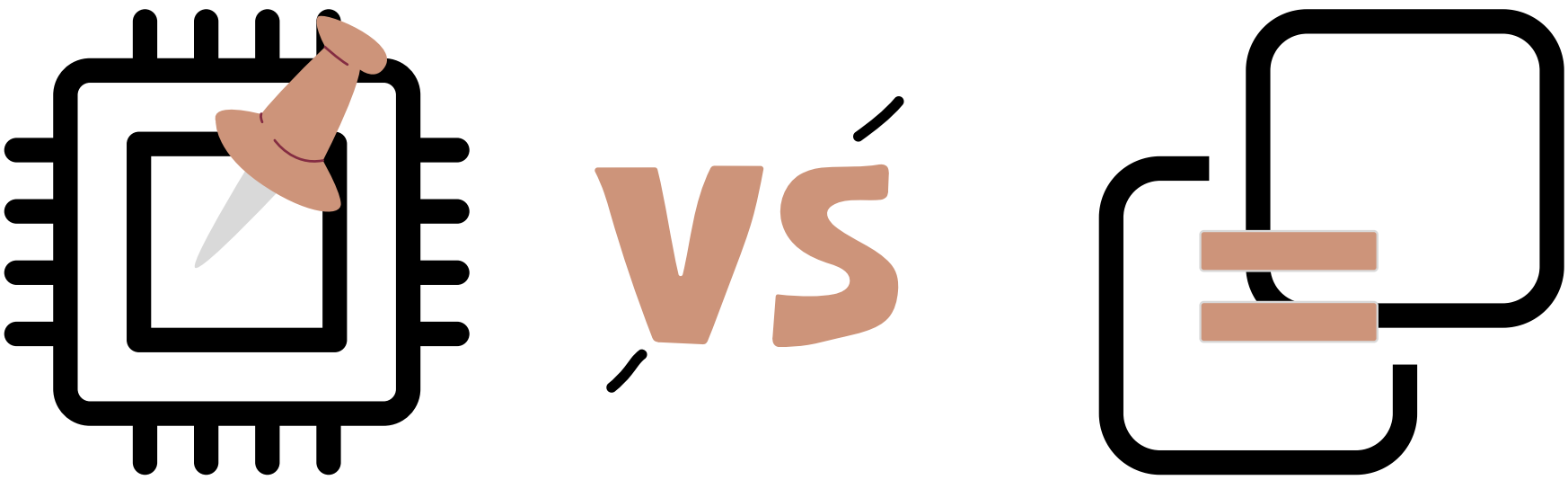
Object Identity

VS

Value Equality



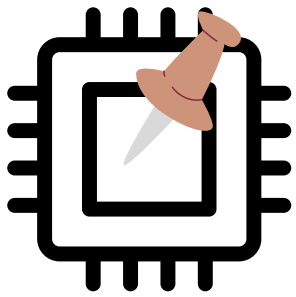
with examples

Georgios Petas

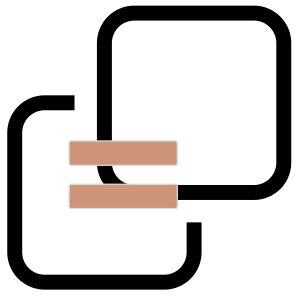
Object Identity vs **Value Equality**

When comparing objects, it's essential to decide whether you want to know if:

- The **objects** are the **same instance** in memory (**Object Identity**).



- The **values** within the objects are **equivalent** (**Value Equality**).



This distinction is important because C# **handles** these **comparisons differently** for **reference types** (classes, delegates, arrays) and **value types** (structs).

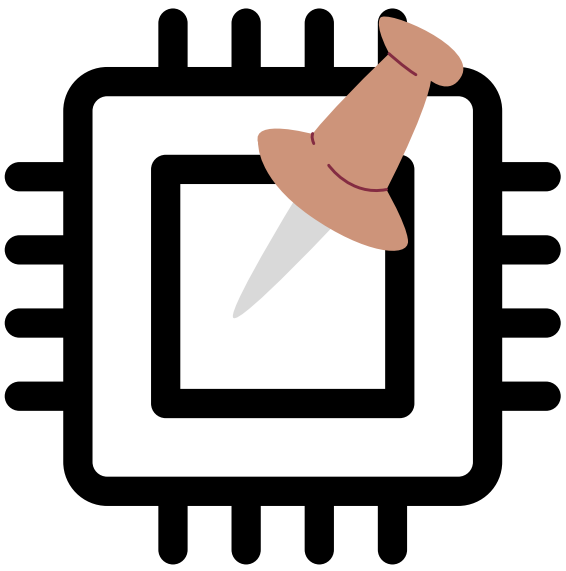
Object Identity

Definition: Checks if **two** object **references point** to the exact **same location** in **memory**.

Use Case: Useful when you need to **ensure** that you're working with the **same instance**, not just an identical copy.

How to Check:

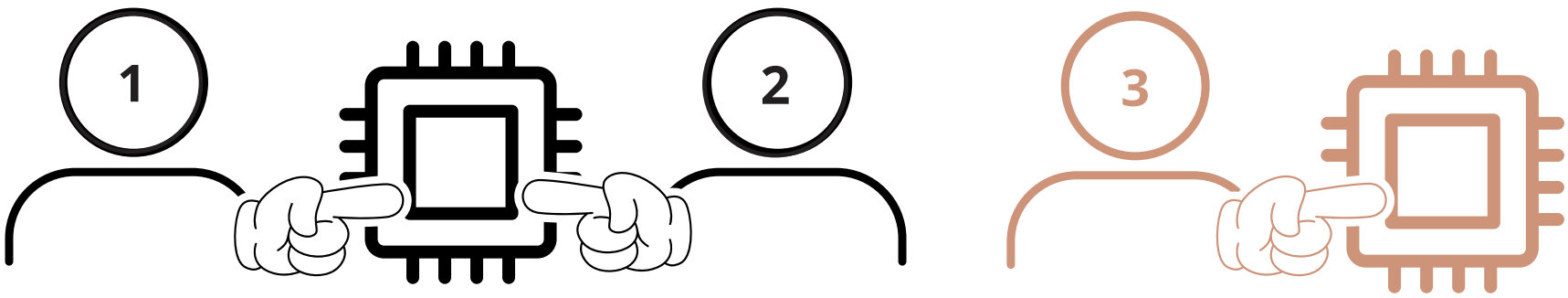
- Use **Object.ReferenceEquals** method.
- Use the **==** operator for **reference types**.



Object Identity - Example

Example

```
class Person
{
    public string Name { get; set; }
    public int Age { get; set; }
}

Person person1 = new Person { Name = "George", Age = 30 };

// Same instance: Both references point to the same object
Person person2 = person1;

// Different instance, same content
Person person3 = new Person { Name = "George", Age = 30 };

// True
Console.WriteLine(Object.ReferenceEquals(person1, person2));

// False
Console.WriteLine(Object.ReferenceEquals(person1, person3));
```



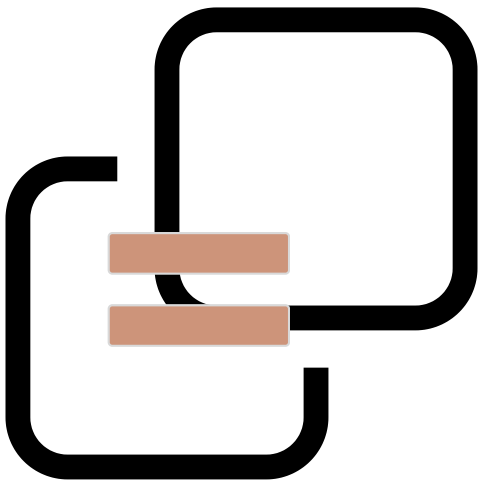
Value Equality

Definition: Checks if the **values** contained in the objects are the **same**, regardless of whether they are different instances.

Use Case: Used when comparing the **data inside objects**, such as comparing two structs with identical fields.

How to Check:

- For **structs**: Use the default ***Equals*** method, which compares field values.
- For **classes**: Override ***Equals*** and ***GetHashCode***, or use **custom implementations**.



Value Equality - Example

Example with Structs

```
public struct Person
{
    public string Name;
    public int Age;

    public Person(string name, int age)
    {
        Name = name;
        Age = age;
    }
}

Person person1 = new Person("George", 30);
Person person2 = new Person("Joe", 42);
person2.Name = "George";
person2.Age = 30;

// True
Console.WriteLine(person2.Equals(person1));
```

Value Equality - Example

Example with Classes

```
class Person : IEquatable<Person>
{
    public string Name { get; set; }
    public int Age { get; set; }

    public override bool Equals(object obj)
    {
        return Equals(obj as Person);
    }

    public bool Equals(Person other)
    {
        if (other == null) return false;
        return Name == other.Name && Age == other.Age;
    }

    public override int GetHashCode()
    {
        return HashCode.Combine(Name, Age);
    }
}

Person person1 = new Person { Name = "George", Age = 30 };
Person person2 = new Person { Name = "George", Age = 30 };

Console.WriteLine(person1.Equals(person2)); // True
```

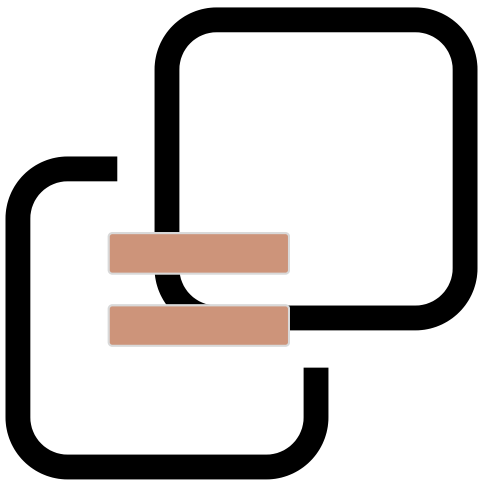
Value Equality with Records

Definition: Records are **reference types** that use **value semantics** for equality.

Use Case: Ideal for comparing objects based on their data rather than their identity.

How to Check:

- Automatically compares field values.



Value Equality with Records - Example

Example

```
public record Person
{
    public string Name { get; init; }
    public int Age { get; init; }
}

Person person1 = new("George", 30);
Person person2 = new("George", 30);

// True
Console.WriteLine(person1 == person2);

// True
Console.WriteLine(person1.Equals(person2));
```

Here, person1 and person2 have the **same** field values, so they are considered **equal** even though they are **different** instances.

Comparison

	Object Identity VS	Value Equality
Definition	Checks if two **references** point to the **same object**.	Compares the **values** inside the **objects**.
Usage	Use **Object.ReferenceEquals** or **==**.	**Override Equals**, **implement IEquatable<T>**, or use custom logic.
Default Behavior for Classes	**Compares reference** (memory location).	Requires **overriding Equals** and **GetHashCode**.
Default Behavior for Structs	**Not applicable** (value types are always copied).	**Compares fields by default.**
Suitable For	Ensuring **objects refer** to the **same instance**.	Checking if **objects** contain **identical data**.
Automatic Handling (Records)	**No automatic handling.**	**Automatically compares all fields.**

That's it!!

If you find this valuable,

follow me for more.

And click the bell in my profile

Georgios Petas